Lab Assignment 4

Task 1: Basic Stack Operations Define a stack structure that can hold integer values. Implement functions to perform basic operations:

- push(int): Add an element to the top of the stack.
- pop(): Remove and return the top element from the stack. If the stack is empty, return a special value or handle the condition gracefully.

Sample Input:

- Push 5.
- Push 10.
- Pop.

Sample Output:

- Top element popped: 10
- Current Stack: 5

Task 2: Checking for Balanced Parentheses Extend your stack operations to include a function that checks if a given string of parentheses is balanced. A string is balanced if each open parenthesis is matched with a closing parenthesis in the correct order.

Sample Input:

- Check balance for "(()())".

Sample Output:

- Is balanced: Yes

Task 3: Implement a Min Stack Modify your stack to support retrieving the minimum element in constant time without removing it. This will likely require additional storage within your stack structure to keep track of the minimum elements.

Sample Input:

- Push 5, Push 3, Push 10, Get Min, Pop, Get Min.

Sample Output:

- Minimum element after pushes: 3

- Minimum element after one pop: 5

Task 4: Reverse a String Using a Stack Implement a function that uses a stack to reverse a given string.

Sample Input:

- Reverse the string "hello".

Sample Output:

- Reversed string: "olleh"

Task 5: Implement a Stack Using a Linked List Instead of using an array, implement your stack using a linked list. This implementation should include the same push and pop operations.

Sample Input:

- Push 1, Push 2, Pop.

Sample Output:

- Top element popped: 2
- Current Stack: 1

Task 6: Design a Stack that Supports Increment Operations Design a stack that supports the following operations:

- push(int x): Push x onto the stack.
- pop(): Remove the element on the top of the stack and return it.
- increment(int k, int val): Increment the bottom k elements of the stack by val. If there are fewer than k elements in the stack, increment all the elements by val.

Sample Input:

- Push 5, Push 2, Push 3, Increment 2, 1, Pop.

Sample Output:

- Current Stack after incrementing bottom 2 elements: 6, 3
- Top element popped: 3
- Current Stack: 6

Task 7: Sort a Stack Implement a function to sort a stack such that the smallest items are on the top. You may use an additional temporary stack, but you should not copy the elements into any other data structure (e.g., array).

Sample Input:

- Push elements 3, 1, 4, 2 into an unsorted stack and sort it. **Sample**

Output:

- Sorted Stack: 1, 2, 3, 4